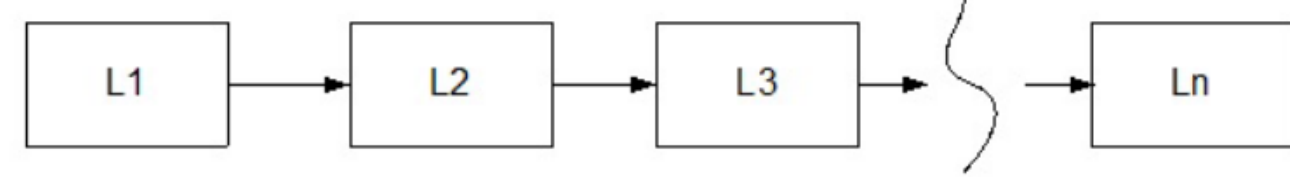


Estructuras linealesListas, pilas y colas

Listas: definición una lista es una estructura de datos lineal flexible, ya que puede crecer a medida que se insertan nuevos elementos o acortarse cuando se borren.

Listas: tipos

- Pilas
- Colas
- Listas propiamente dichas

Listas: operaciones

Alta

Baja

Consulta

Modificación

Pilas: características

Alta al final

Baja al final

Consulta al final

Pilas: usos

- Funciones recursivas, por ejemplo la función factorial recursiva.
- Editores de texto, cuando uno presiona el botón de deshacer vuelve al estado anterior inmediato, esos cambios se guardan en una pila.
- la pila (stack) que usa la memoria RAM para almacenar variables locales.
- Los navegadores de internet, cuando uno está en un sitio y presiona un link, va a otro sitio, cuando presiona la flecha de regresar vuelve a la página anterior.
los sitios visitados los guarda en una pila.

Colas: características

Alta al final

Baja al principio

Cola: usos

- Cuando vamos a un banco y sacamos un número nos coloca al final de la cola. Los empleados van atendiendo a las personas que llegaron antes. Cada vez que atienden a una persona decimos que sale de la cola.
- Varias máquinas conectadas a una impresora, en donde los distintos usuarios envían a imprimir archivos desde sus terminales. la solicitud ingresa a una cola en donde se van procesando de una en una cada solicitud.

- Nota: una cola respeta siempre el orden de llegada, aunque hay algunas variantes en donde se pueden determinar distintas prioridades que alteren dicho orden.

Por ejemplo, una solicitud de impresión tendrá prioridad más alta que el proceso de escaneo de todo el disco en busca de virus. Un cliente premium de un banco tendrá prioridad más alta que un cliente que no lo es, por lo que será atendido en primer lugar aunque haya llegado después.

En estos casos, tenemos una estructura que llamamos cola de prioridades que veremos más adelante.

Listas: características

Alta sin restricciones

Baja " "

Consulta " "

Lista: usos

Uso genérico de colección de datos:

- lista de películas
- lista de empleados
- lista de estudiantes

Pilas: implementaciones

- Estática: se da un tamaño fijo como tope al principio que no puede exceder
- Dinámica: va creciendo a medida que van ingresando elementos, utilizando memoria dinámica y decreciendo a medida que se los borra.

Implementación estática

```
public class Pe {  
    private final int MAX = 10;  
    private int[] datos;  
    private int ultimo;  
  
    // constructor  
    public Pe() {  
        datos = new int[MAX];  
        ultimo = 0;  
    }  
}
```

```
public void agregar(int dato) {  
    datos[ultimo] = dato;  
    ultimo++;  
}  
  
public int sacar() {  
    ultimo--;  
    return datos[ultimo];  
}  
  
public boolean vacia() {  
    return (ultimo == 0);  
}}
```

uso:

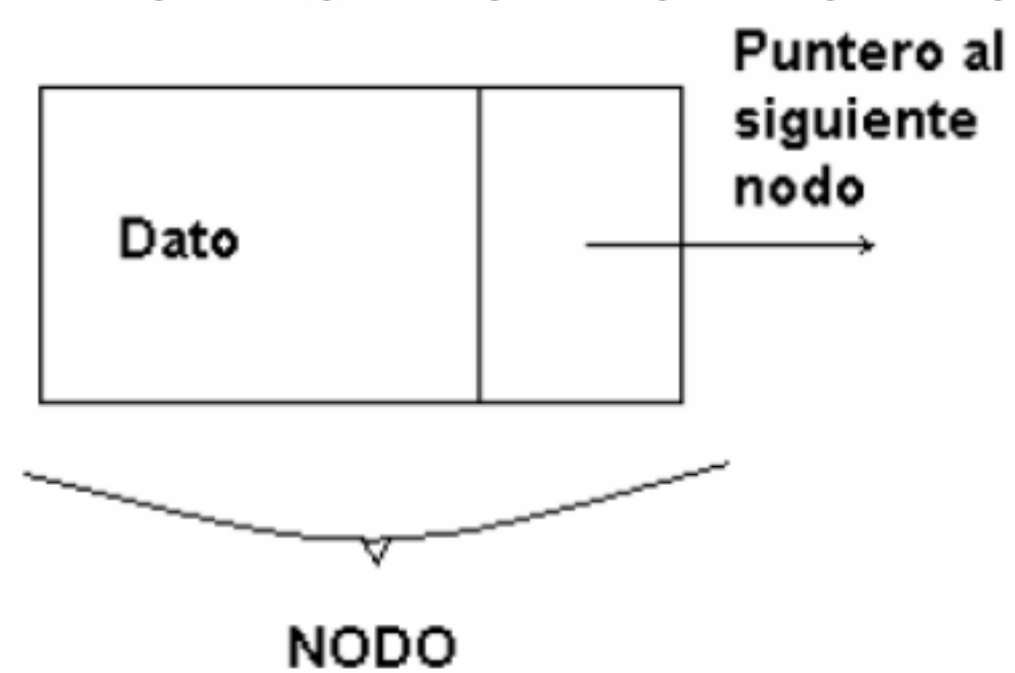
```
public static void main(String[] args) {  
    Pe p = new Pe();  
    p.agregar(5);  
    p.agregar(8);  
    p.agregar(3);  
    while (!p.vacia()) {  
        System.out.println(p.sacar());  
    }  
}
```


Pila: implementación dinámica

Enlaza nodos.

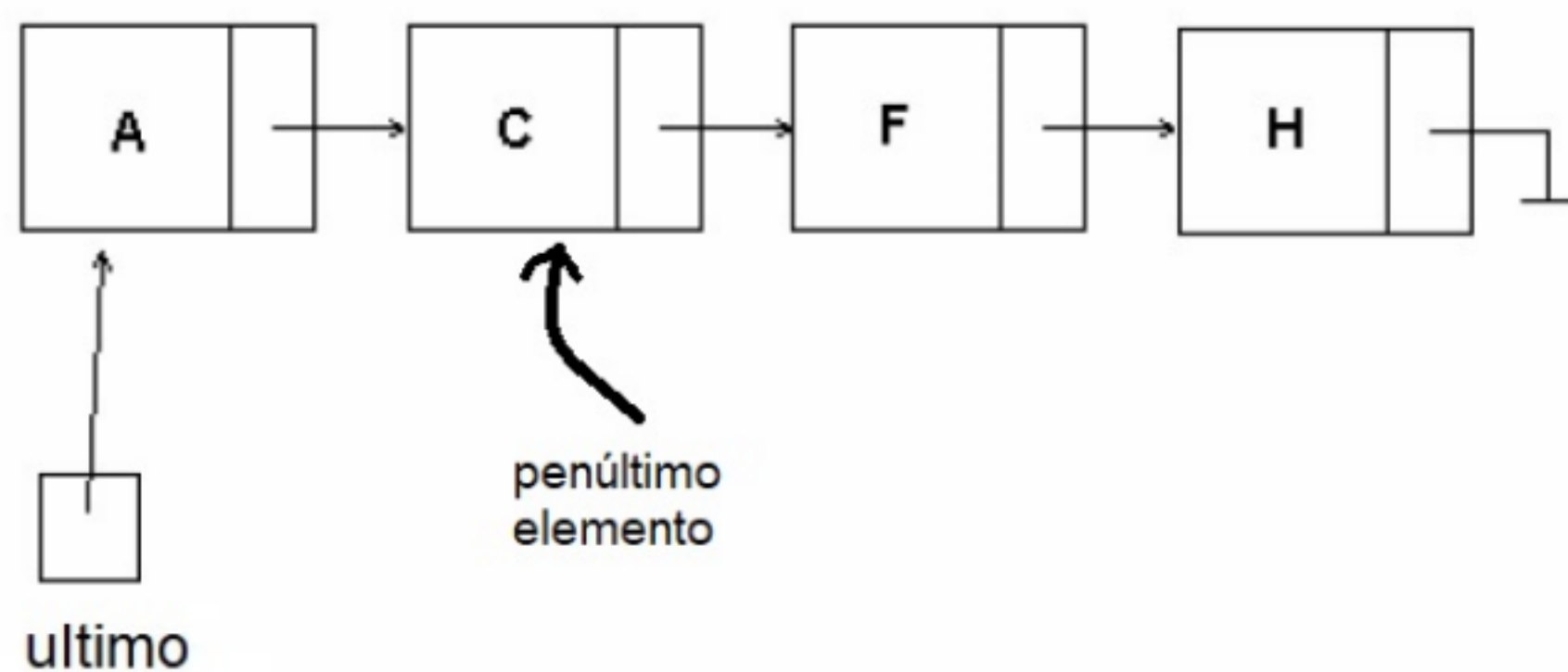
Un nodo tendrá el dato en sí y un puntero a otro nodo.

Se enlazan como si fueran eslabones de una cadena.

Implementación dinámica

El objeto Pila tendrá un puntero al último nodo con el último elemento ingresado.

Este nodo enlazará con el anterior, el que a su vez enlazará con su anterior, etc.

Nodo

```
public class Nodo {
    private int dato;
    private Nodo sig;
    // constructor
    public Nodo(int d) {
        dato = d;
        sig = null;
    }
    public int getDato() {
        return dato;
    }
}
```

```
public void setDato(int dato) {
    this.dato = dato;
}
public Nodo getSig() {
    return sig;
}
public void setSig(Nodo sig) {
    this.sig = sig;
}
}
```

Pila: implementación dinámica

```
public class Pila {  
    private Nodo ultimo;  
    public Pila() {  
        ultimo = null;  
    }  
    public void agregar(int dato) {  
        Nodo nuevo = new Nodo(dato);  
        nuevo.setSig(ultimo);  
        ultimo = nuevo;  
    }  
}
```

```
public int desapilar() {  
    int devolver = ultimo.getData();  
    ultimo = ultimo.getSig();  
    return devolver;  
}  
  
public boolean vacia() {  
    return (ultimo == null);  
}  
}
```